

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	SYED MUHAFIZ A	Reg. No.:	192472282
Section / Batch:	SLOT-C	Date:	22-08-2026

Code of Conduct

I SYED MUHAFIZ A / 192472282 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: muhafiz

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A — Pseudocode Writing Task (Algorithm Design): Answers

Q1. Reference Resolution — Entity Linking Algorithm

Text: *"Ravi met Arun at the library. He borrowed a book and later returned it."*

Algorithm (Pseudocode)

```
ALGORITHM ReferenceResolution(text):
  Sentences <- SplitIntoSentences(text)
  EntityList <- [] // discourse entity list, in order of
  mention
  ResolvedChains <- {}

  FOR each sentence S in Sentences:
    Tokens <- Tokenize(S)
    FOR each token T in Tokens:
      IF IsNounPhrase(T) THEN
        features <- {gender: InferGender(T), number: InferNumber(T),
                     animacy: InferAnimacy(T), role: SyntacticRole(T)}
        EntityList.append({mention: T, features, sentIdx, position})

      ELSE IF IsPronoun(T) THEN
        // Step 1: Hard filter -- agreement
        candidates <- Filter(EntityList, e -> Agrees(e.features, T))

        // Step 2: Hard filter -- semantic compatibility
        candidates <- Filter(candidates, e -> SemanticFit(e, T, S))

        // Step 3: Soft preference -- subject/grammatical-role parallelism
        candidates <- RankBySubjectParallelism(candidates, T)

        // Step 4: Soft preference -- recency
        antecedent <- MostRecent(candidates)

        Link(T, antecedent)
        ResolvedChains[antecedent].append(T)

  RETURN ResolvedChains
```

Applying the Algorithm to the Text

Entity list built while scanning: E1 = Ravi (subject of S1, male, singular, animate), E2 = Arun (object of S1, male, singular, animate), E3 = the library (location, inanimate), E4 = a book (object, inanimate, introduced in S2).

- "He" (subject of S2) → agreement passes for both Ravi and Arun (both male, singular). Subject/grammatical-role parallelism prefers the antecedent that also occupied the subject position in the previous clause → Ravi (subject of "met"). → He = Ravi.
- "it" (object of "returned") → agreement candidates are the library and the book (both non-human, so "it" is grammatically compatible with either). Semantic-fit filtering applies: the verbs "borrowed"/"returned" select a portable, transferable object as their argument — a library (a fixed location) cannot be borrowed or returned, whereas a book can. → it = a book.

Resolved Discourse

"Ravi met Arun at the library. Ravi [He] borrowed a book and later returned the book [it]."

Validation — How Discourse Context Identifies Antecedents

The resolution is validated by tracing each decision back to explicit discourse-context evidence rather than a guess: (1) ϕ -feature agreement narrows the candidate set using surface grammar alone; (2) selectional/semantic compatibility uses the verb's argument requirements ("borrow/return X" needs a transferable object) to eliminate grammatically legal but semantically implausible antecedents such as "the library"; (3) subject-parallelism uses the observation that English discourse frequently continues describing the most recently topicalized (subject-position) entity, which correctly picks Ravi over Arun for "He"; and (4) recency acts as the final tie-breaker when the previous constraints under-determine the answer. Because every step is grounded in a specific, checkable property of the sentence (grammatical role, verb semantics, ϕ -features), the resolution process is explainable and can be independently verified against the source text.

Q2. Text Coherence & Discourse Structure — Coherence Analysis Algorithm

Discourse: "The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online."

Algorithm (Pseudocode)

```
ALGORITHM DiscourseStructureAnalysis(text):
  Sentences <- SplitIntoSentences(text)
  Relations <- []
  CueLexicon <- {
    "because", "after", "since"      -> CAUSE,
    "therefore", "so", "as a result" -> RESULT,      // Cause-Effect (marks the
effect)
    "then", "next", "later"          -> SEQUENCE,
    "for example", "in particular"    -> ELABORATION,
    "but", "however"                 -> CONTRAST
  }

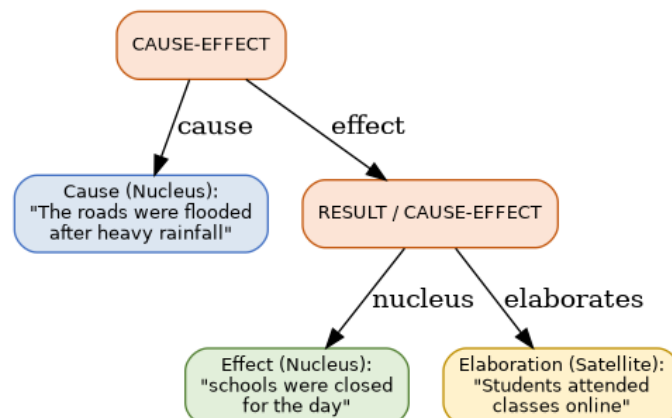
  FOR i = 2 to length(Sentences):
    cue <- DetectDiscourseCue(Sentences[i], CueLexicon)
    IF cue is found THEN
      relation <- CueLexicon[cue]
    ELSE
      relation <- InferImplicitRelation(Sentences[i-1], Sentences[i])
      // e.g., shared entities + temporal/topical continuation
      // with no causal cue => ELABORATION by default

    Relations.append({from: Sentences[i-1], to: Sentences[i], type: relation})

  Tree <- BuildDiscourseTree(Relations)  // nucleus/satellite assignment per RST
  RETURN Tree
```

Applying the Algorithm to the Discourse

- S1 → S2: the cue word "Therefore" at the start of S2 is directly matched in the CueLexicon to RESULT (a Cause-Effect relation) — the flooding (S1) is the Cause, and the school closure (S2) is the Effect.
- S2 → S3: no explicit discourse cue connects S3 to S2. InferImplicitRelation checks shared entities/topic (schools/education) and finds S3 supplies additional detail about how education continued despite the closure — classified as ELABORATION (with a secondary Result flavour, since going online is itself a consequence of the closure).



Justification — How These Relations Maintain Coherence

Coherence is not just about topical continuity (all three sentences mention roads/schools/students in a shared "disrupted school day" scenario) — it requires the reader to recover the logical connections between events. Explicitly labelling $S1 \rightarrow S2$ as Cause-Effect lets a reader (or a downstream system, e.g., a summarizer or QA module) correctly answer "why were schools closed?" from the text. Labelling $S2 \rightarrow S3$ as Elaboration correctly signals that $S3$ is not an independent new event but a detail subordinate to the closure event, preventing a system from, say, treating "students attended classes online" as unrelated or as a fourth cause. Together, the two relations form a coherent causal chain (rainfall $\rightarrow$ flooding $\rightarrow$ closure $\rightarrow$ online classes) rather than three disconnected facts, which is exactly what discourse-relation labelling is meant to make explicit and machine-checkable.

Q3. Conversational Agent — Dialogue Act / Intent Recognition Algorithm

Conversation:

User: "Can you book a train ticket for me?" Agent: "Sure, where would you like to travel?" User: "I want to go to Chennai." Agent: "Your ticket has been booked."

Algorithm (Pseudocode)

```
ALGORITHM ClassifyDialogueAct(utterance):
  u <- Normalize(utterance)           // lowercase, strip punctuation for
  matching

  IF MatchesPattern(u, ["can you", "could you", "please"])
    AND ContainsActionVerb(u) THEN
    RETURN REQUEST

  IF StartsWithWhWord(u) OR IsInvertedQuestionForm(u) THEN
    act <- QUESTION
    IF ContainsAcknowledgement(u, ["sure", "okay", "yes"]) THEN
      act <- CONFIRMATION + QUESTION    // compound act
    RETURN act

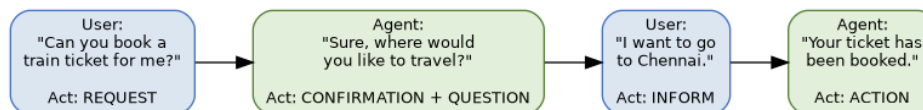
  IF ContainsCompletionMarker(u, ["booked", "done", "confirmed", "has been"]) THEN
    RETURN ACTION                      // system reports a completed action

  IF IsDeclarative(u) AND ContainsFirstPersonStatement(u) THEN
    RETURN INFORM

  RETURN OTHER

ALGORITHM AnalyzeConversation(turns):
  Sequence <- []
  FOR each utterance u in turns:
    Sequence.append(ClassifyDialogueAct(u))
  RETURN Sequence
```

Applying the Algorithm — Classification



Dialogue-act sequence produced by the algorithm.

- "Can you book a train ticket for me?" → matches the Request pattern ("Can you" + action verb "book") → REQUEST.
- "Sure, where would you like to travel?" → "Sure" is an acknowledgement marker and the clause is a wh-question → CONFIRMATION + QUESTION (compound act).
- "I want to go to Chennai." → declarative, first-person statement of a preference/fact → INFORM.
- "Your ticket has been booked." → contains the completion marker "has been booked", reporting a completed system action rather than asking or requesting anything → ACTION.

Dialogue-act sequence: REQUEST → (CONFIRMATION + QUESTION) → INFORM → ACTION.

Evaluation — How the Acts Help Interpret User Intentions

The dialogue-act sequence maps directly onto a slot-filling task frame that the agent must complete to satisfy the user's goal: the REQUEST triggers task/intent detection ("book_train_ticket"), the agent's QUESTION identifies a missing required slot (destination), the user's INFORM fills that slot with a value ("Chennai"), and the agent's ACTION executes the task and reports task completion. Recognizing the act type at each turn — rather than only the literal words — lets the agent decide the correct system behaviour at each step (initiate a task vs. request a missing parameter vs. store a value vs. execute and confirm), which is exactly what a task-oriented dialogue manager needs to interpret user intent correctly across multiple turns instead of treating each utterance in isolation.

Q4. Language Generation — NLG Pipeline / Surface Realization Algorithm

Semantic input: *Action: Buy, Agent: Student, Object: Book, Tense: Past*

Algorithm (Pseudocode)

```
ALGORITHM SurfaceRealization(semanticInput):
  // semanticInput = {Action, Agent, Object, Tense, ...}

  // Step 1: Lexical selection
  verbLexeme    <- LexicalSelect(semanticInput.Action)    // Buy -> "buy"
  subjectLexeme <- LexicalSelect(semanticInput.Agent)     // Student -> "student"
  objectLexeme  <- LexicalSelect(semanticInput.Object)    // Book -> "book"

  // Step 2: Sentence structuring (map roles to syntactic positions)
  structure <- AssignSyntacticRoles(Agent -> SUBJECT, Action -> VERB, Object ->
OBJECT)
  order     <- ApplyWordOrderRules(LANGUAGE = English)    // SVO

  // Step 3: Morphological realization
  verbForm <- InflectVerb(verbLexeme, semanticInput.Tense) // buy -> bought
(irregular past)

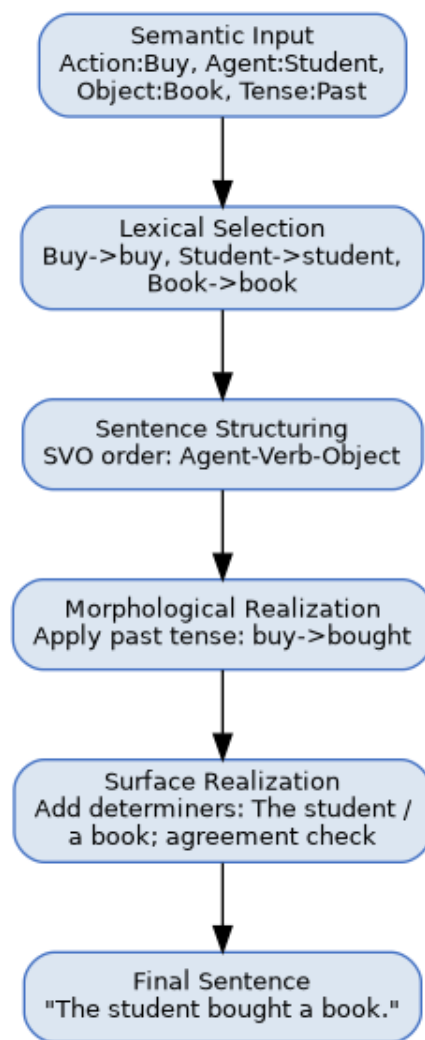
  // Step 4: Referring-expression / determiner generation
  subjectPhrase <- AddDeterminer(subjectLexeme, definiteness = DEFINITE) // "The
student"
  objectPhrase  <- AddDeterminer(objectLexeme, definiteness = INDEFINITE) // "a
book"

  // Step 5: Linearization and agreement
  sentence <- OrderConstituents(order, subjectPhrase, verbForm, objectPhrase)
sentence <- CheckSubjectVerbAgreement(sentence)
sentence <- CapitalizeAndPunctuate(sentence)

RETURN sentence
```

Generated Output

"The student bought a book."



NLG pipeline: semantic input → lexical selection → structuring → morphological realization → surface realization.

Explanation of the Stages

- Lexical selection: each semantic role/concept is mapped to a concrete English lexeme — the predicate Buy is mapped to the verb "buy", the Agent Student to the noun "student", the Object Book to the noun "book".
- Sentence structuring: the semantic roles are assigned to syntactic functions (Agent → grammatical subject, Object → grammatical object) and ordered according to English's canonical Subject–Verb–Object word order.
- Surface realization: the abstract structure is turned into concrete, inflected, punctuated text — the verb is morphologically inflected for past tense (the irregular form "bought", not "**bued"), determiners are inserted (definite "the" for the already-established Agent, indefinite "a" for the newly introduced Object), and subject–verb agreement and capitalization/punctuation are applied.

Validation of Grammatical Correctness

"The student bought a book." is checked against standard English grammar rules: (1) subject–verb agreement — the past tense form "bought" is invariant for number/person, so it is compatible with the singular subject "student"; (2) correct irregular past-tense morphology — "buy" correctly realizes as "bought", not the regular "**bued"; (3) correct determiner use — "the" for a definite, already-known agent and "a" for an indefinite, newly mentioned object, matching standard article-choice conventions; (4) correct SVO constituent order for an English declarative sentence. All four checks pass, confirming the output is grammatically well-formed.

Q5. Interlingua-based + Statistical Machine Translation — Translation Workflow Algorithm

Sentence to translate: *"The boy is playing football."*

Algorithm (Pseudocode)

```
ALGORITHM HybridInterlinguaStatisticalMT(sourceSentence, targetLanguage):

    // Step 1: Analyze the source sentence
    tokens <- Tokenize(sourceSentence)
    parseTree <- SyntacticParse(tokens)           // POS tags + dependency structure
    roles <- ExtractSemanticRoles(parseTree)       // Agent, Action, Object, Tense,
Aspect

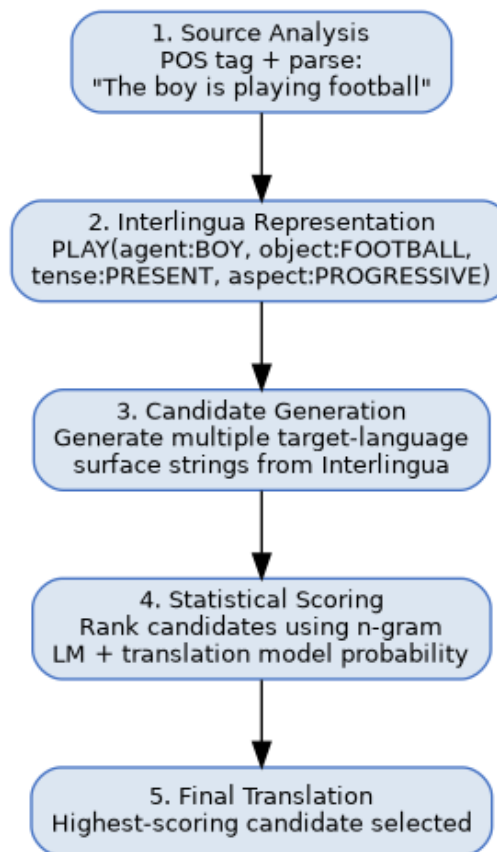
    // Step 2: Build an Interlingua (language-independent) representation
    IL <- BuildInterlingua(roles)
    // e.g., PLAY(agent: BOY, object: FOOTBALL, tense: PRESENT, aspect:
PROGRESSIVE)

    // Step 3: Generate candidate translations from the Interlingua
    Candidates <- []
    FOR each generationRule in TargetGrammarRules(targetLanguage):
        candidate <- GenerateSurfaceForm(IL, generationRule)
        Candidates.append(candidate)

    // Step 4: Apply statistical scoring to rank candidates
    FOR each c in Candidates:
        c.score <- LanguageModelProb(c) * TranslationModelProb(c, sourceSentence)
        // n-gram fluency score x adequacy/alignment score from parallel
corpus

    // Step 5: Select and produce the final translation
    best <- ArgMax(Candidates, key = c -> c.score)
    RETURN best
```

Applying the Algorithm to "The boy is playing football."



Hybrid Interlingua + statistical translation workflow.

1. Source analysis: parsing gives Subject = "the boy" (definite, singular), Verb = "is playing" (present progressive), Object = "football".
2. Interlingua representation: PLAY(agent: BOY, object: FOOTBALL, tense: PRESENT, aspect: PROGRESSIVE) — a language-neutral semantic frame with no English-specific words or word order.
3. Candidate target-language translations are generated from this frame (Tamil target, illustrative):
 - C1: "பையன் கால்பந்து விளையாடுகிறான்." (paiyaṇ kālpaṇḍhu viḷaiyaḍugirāṇ)
 - C2: "சிறுவன் கால்பந்து ஆடுகிறான்." (siruvaṇ kālpaṇḍhu āḍugirāṇ) — alternate lexical choices for "boy" and "play"
4. Statistical scoring: a language-model probability (how fluent/frequent the word sequence is in target-language text) combined with a translation-model probability (how well it aligns with the source, from parallel-corpus statistics) is computed for each candidate. C1 scores higher because "கால்பந்து விளையாடு" ("play football") is a much more frequent, natural collocation in Tamil corpora than the alternate verb choice in C2.
5. Final translated sentence: "பையன் கால்பந்து விளையாடுகிறான்."

Evaluation — Why the Hybrid Improves Quality and Reduces Ambiguity

The Interlingua stage forces the system to first commit to an explicit, unambiguous meaning representation (who is doing what, to what, in what tense/aspect) before any target words are chosen. This removes structural and lexical ambiguity that a direct source-to-target statistical system might otherwise carry over or mistranslate (e.g., misassigning tense/aspect or verb-argument roles). The statistical scoring stage then contributes exactly what a purely symbolic Interlingua-to-text generator lacks: fluency and naturalness, since multiple candidates can be semantically correct but only some read as idiomatic, corpus-frequent target-language text. By combining the two — Interlingua for semantic correctness and completeness, statistics for fluent, natural surface choice — the hybrid approach produces translations that are both meaning-preserving and natural-sounding, addressing the

two failure modes (meaning loss and unnatural phrasing) that pure statistical MT and pure rule-based Interlingua MT are each individually prone to.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____